

RDL TECHNOLOGIES PVT LTD

Internship Project Report

Edge AI-Based Solar Panel Health Monitoring System

Using TinyML, ESP32 and IoT Dashboard Integration

Submitted by

Ashlesh Y Saliyan

NMAM Institute of technology

USN - NNM24EC028

Internship Duration: 01/06/2026 to 31/07/2026

Under the Guidance of: Ms. Sharanya, RDL Technologies Pvt Ltd

Domain: Internet of Things

Certificate

This is to certify that Ashlesh Y Saliyan, a student of NMAM institute of technology , has successfully completed an internship at RDL Technologies Pvt Ltd from 01/06/2026 to 31/07/2026, during which the project titled "Edge AI-Based Solar Panel Health Monitoring System Using TinyML, ESP32 and IoT Dashboard Integration" was designed, developed, and tested under the supervision of Ms. Sharanya.

The work presented in this report is an authentic record of the intern's own work carried out during the internship period.



Signature of Mentor

Ms. Sharanya, RDL Technologies Pvt Ltd

Acknowledgement

I would like to express my sincere gratitude to RDL Technologies Pvt Ltd for providing me the opportunity to undertake this internship and work on a hands-on Edge AI and IoT project. I am especially thankful to Ms.Sharanya for their continuous guidance, technical support, and valuable feedback throughout the development of this system. I would also like to thank my teammates Anish, Abhishek, Ayush and Preetham for collaborating on the Node-RED dashboard integration, and my institution, NMAM Institute of technology, for supporting this internship as part of my academic curriculum.

Abstract

Conventional solar panel monitoring systems typically track only voltage and current, relying on simple threshold checks to flag underperformance. This approach cannot distinguish between different root causes of reduced output — such as dust accumulation, partial shading, overheating, or an outright electrical fault — since all of them can produce a similar drop in readings. This project addresses that limitation by building a low-cost Edge AI system that uses a machine learning model, trained and deployed via Edge Impulse, to classify the solar panel's operating condition directly on an ESP32 microcontroller.

The system reads voltage, current, panel temperature, and ambient light intensity using a resistive voltage-divider sensor, a DS18B20 digital temperature sensor, and a light-dependent resistor (LDR). These readings are fed into a TinyML neural network classifier trained on six operating conditions: Healthy, Dust Accumulation, Partial Shading, No Sunlight, Overheating, and Fault. The classification result, along with live sensor data, is published over MQTT (via HiveMQ Cloud) to a Node-RED dashboard that displays real-time parameters, the AI-predicted health status, and overall system status.

The final deployed model achieves 89.17% accuracy on held-out test data, with four of the six classes classified with 100% accuracy. This report documents the complete system design — hardware architecture, sensor integration, the Edge Impulse machine learning pipeline, and full-stack integration with a cloud-connected IoT dashboard.

Table of Contents

1. Introduction
 2. Literature Survey / Existing Systems
 3. System Architecture
 4. Hardware Design and Implementation
 5. Machine Learning Pipeline (Edge Impulse)
 6. Software Integration
 7. Results and Testing
 8. Conclusion and Future Scope
 9. References
- Appendix A: Key Source Code Excerpts
- Appendix B: Node-RED Function Node Source Code

1. Introduction

1.1 Background

Solar panels are widely used for renewable energy generation, but their efficiency degrades over time due to factors such as dust accumulation, partial shading from surrounding objects, overheating in direct sunlight, and electrical or connection faults. Manual inspection of panel health is labour-intensive and typically reactive rather than preventive. Basic IoT monitoring solutions that only track voltage and current can detect that something is wrong, but not what is wrong, since multiple distinct fault conditions can produce similar drops in electrical output.

1.2 Problem Statement

There is a need for a low-cost, on-device intelligent monitoring system that can not only measure a solar panel's electrical output but also classify the underlying cause of any performance degradation in real time, without depending on constant cloud connectivity for inference.

1.3 Objectives

- To design a multi-sensor data acquisition system for a solar panel using an ESP32 microcontroller.
- To collect a labelled dataset representing six distinct panel operating conditions.
- To train a TinyML classification model using Edge Impulse and deploy it directly on the ESP32 for on-device inference.
- To transmit live sensor data and AI predictions to a remote dashboard using MQTT.
- To build an interactive Node-RED dashboard for real-time visualization of panel health.

1.4 Scope of the Project

The project covers the complete pipeline from sensor-level hardware integration through to a cloud-connected dashboard: sensor circuit design, dataset collection through physical simulation of fault conditions, model training in Edge Impulse, embedded deployment of the trained model, and MQTT-based integration with a Node-RED dashboard accessible from a separate machine.

2. Literature Survey / Existing Systems

Most commercially available solar monitoring systems (e.g., basic IoT dataloggers) report raw voltage, current, and power values on a dashboard, leaving fault diagnosis to the user's interpretation. Some advanced commercial systems use cloud-based analytics to detect anomalies, but this requires a constant internet connection and introduces latency between fault occurrence and detection. TinyML — the practice of running machine learning inference directly on microcontrollers — offers an alternative: fault classification can happen on the device itself, in milliseconds, without depending on a cloud round-trip. This project applies that approach specifically to solar panel health monitoring, using Edge Impulse as the TinyML development platform and an ESP32 as the target hardware, which is well suited given its sufficient RAM/flash for small neural networks and its built-in WiFi for dashboard connectivity.

3. System Architecture

3.1 Overview

The system follows a sensor-to-dashboard pipeline: the solar panel's electrical and environmental parameters are read by the ESP32, processed and classified on-device using the trained Edge Impulse model, and the resulting data (raw readings plus AI prediction) is published over MQTT to a Node-RED dashboard for visualization.

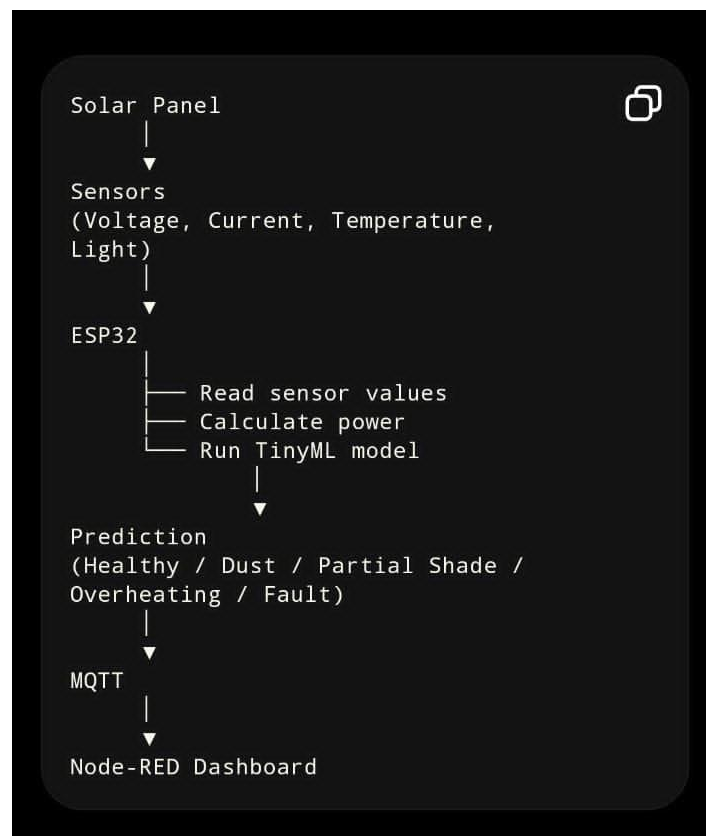


Figure 3.1: System architecture and data flow, from solar panel sensing through to the Node-RED dashboard.

3.2 Hardware Components

Component	Purpose
ESP32 DevKit V1	Main microcontroller: sensor reading, AI inference, WiFi/MQTT communication
Voltage Sensor Module (0–25V, resistive divider)	Scales down panel voltage to a safe 0–3.3V range for the ADC
DS18B20	Digital temperature sensor mounted on the panel's rear surface
LDR (Light Dependent Resistor)	Measures ambient light intensity, independent of the panel's own output
Small solar panel (d.light branded)	Test panel used as the monitored device
Breadboard and jumper wires	Prototyping and sensor interconnection

3.3 Software and Tools Used

Tool / Platform	Role
Arduino IDE 2.3.2	Firmware development and upload for the ESP32
Edge Impulse Studio	Dataset management, model training, and Arduino library export
HiveMQ Cloud	TLS-secured MQTT broker connecting the ESP32 to the dashboard
Node-RED + node-red-dashboard	Real-time web dashboard for visualizing sensor data and AI predictions
PubSubClient / WiFiClientSecure	Arduino libraries for MQTT communication over TLS

4. Hardware Design and Implementation

4.1 Sensor Mounting

The DS18B20 temperature sensor was mounted directly onto the exposed rear diffuser surface of the solar panel using thermal adhesive to ensure good thermal contact, allowing the sensor to accurately track the panel's surface temperature during operation.



Figure 4.1: Rear view of the solar panel showing the mounting surface used for the temperature sensor.

4.2 Circuit Wiring

All sensors were interfaced with the ESP32 on a breadboard. The voltage sensor module connects to the panel's output leads via screw terminals, and its scaled-down signal output feeds directly into an ESP32 ADC pin.



Figure 4.2: Voltage sensor module used in the circuit, connected via screw terminals to the panel and via a 3-pin header to the ESP32.

4.3 Voltage and Current Sensing

The voltage sensor module divides the panel's output voltage by a fixed ratio of 5 (based on $30\text{k}\Omega/7.5\text{k}\Omega$ divider resistors), bringing it safely within the ESP32 ADC's 0–3.3V input range. The scaled ADC reading is multiplied back by this ratio in firmware to recover the actual panel voltage.

Current is derived from the measured voltage using the panel's rated specification (5V, 360mA) as a linear reference, scaling proportionally with voltage and capped at the panel's rated maximum. Combined with voltage and temperature, and with the LDR providing an independent ambient-light reading (see Section 5.1), this feature set was sufficient to reliably distinguish between all six trained operating conditions.

5. Machine Learning Pipeline (Edge Impulse)

5.1 Data Collection Methodology

Six operating conditions were defined for classification: Healthy, Dust Accumulation, Partial Shading, No Sunlight, Overheating, and Fault. Each was physically simulated and logged using a custom Arduino sketch that allowed a text label to be set over Serial before each logging session, so that incoming sensor rows were automatically tagged.

A key design decision was placing the LDR separately from the solar panel rather than directly on it, so that it measures ambient light independent of the panel's actual electrical output. This created a distinguishing signature between otherwise similar-looking fault conditions:

Condition	Simulation Method	Expected Signature
Healthy	Panel in direct, unobstructed light	High voltage, ambient light bright
Dust	Panel covered with a thin translucent sheet; LDR left exposed	Reduced voltage despite bright ambient light
Partial Shade	Both panel and LDR partially covered together	Moderate voltage, moderate ambient light
No Sunlight	Both panel and LDR fully covered / light source off	Near-zero voltage, very low ambient light
Fault	Panel or sensor disconnected, circuit opened, LDR left exposed	Near-zero voltage despite bright ambient light
Overheating	DS18B20 warmed directly while panel remains lit	Elevated temperature, slightly reduced voltage

This approach allowed the two hardest-to-distinguish pairs — Fault vs. No Sunlight, and Dust vs. Healthy — to remain separable using the light-sensor / panel-output relationship, even without a fully independent current measurement.

5.2 Dataset Preparation

Data was logged at 1 sample per second, with roughly 100–150 samples collected per class, then merged into a single labelled dataset of 688 rows with the schema voltage, current, temperature, light, label. During upload to Edge Impulse's CSV Wizard, exact-duplicate rows (common in static conditions such as Fault and No Sunlight, where readings stayed constant for extended periods) were automatically filtered, leaving 598 unique training samples split 80/20 into training and test sets.

5.3 Model Design

Since the input consists of independent per-second sensor readings rather than a continuous time-series waveform, a Flatten processing block was used, followed by a Classification (Neural Network) learning block with two dense hidden layers (20 and 10 neurons) and a 6-class output layer, trained for 500 cycles with class auto-weighting enabled to account for minor class imbalance in the dataset.

5.4 Model Performance

The final trained model achieved 93.8% accuracy on the validation set and 89.17% accuracy on completely held-out test data, with an AUC of 0.99. Four of the six classes — Fault, No Sunlight, Overheating, and Partial Shade — were classified with 100% accuracy; Dust and Healthy, which share more overlapping sensor signatures, achieved 73.9% and 70.8% respectively.

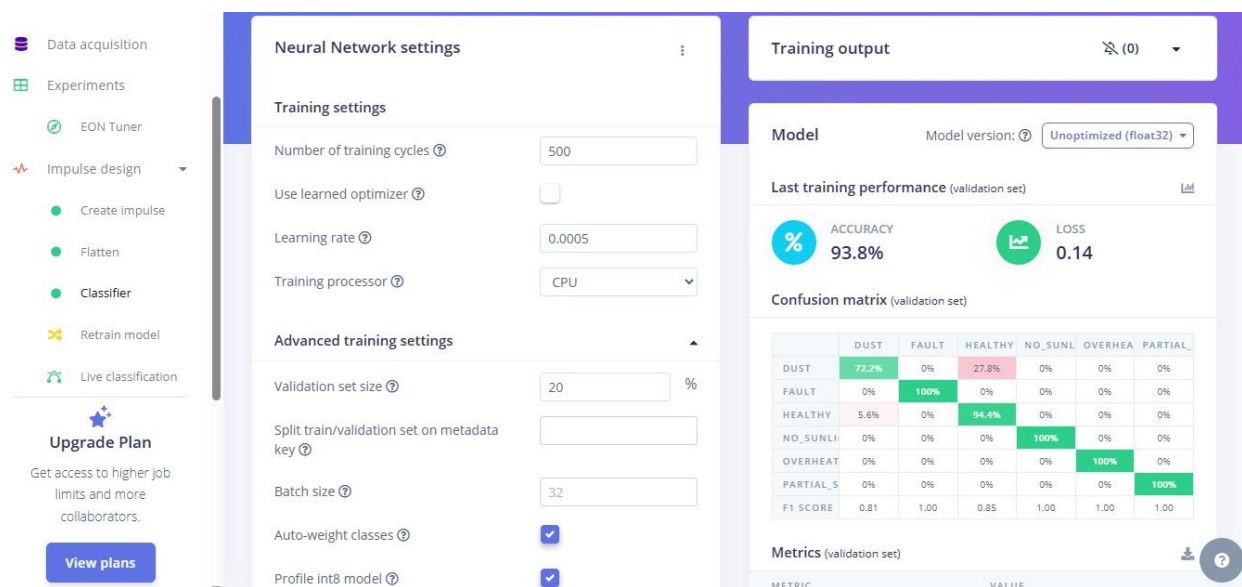


Figure 5.1: Final model configuration and validation performance — 93.8% accuracy, with only Dust and Healthy below 90%.

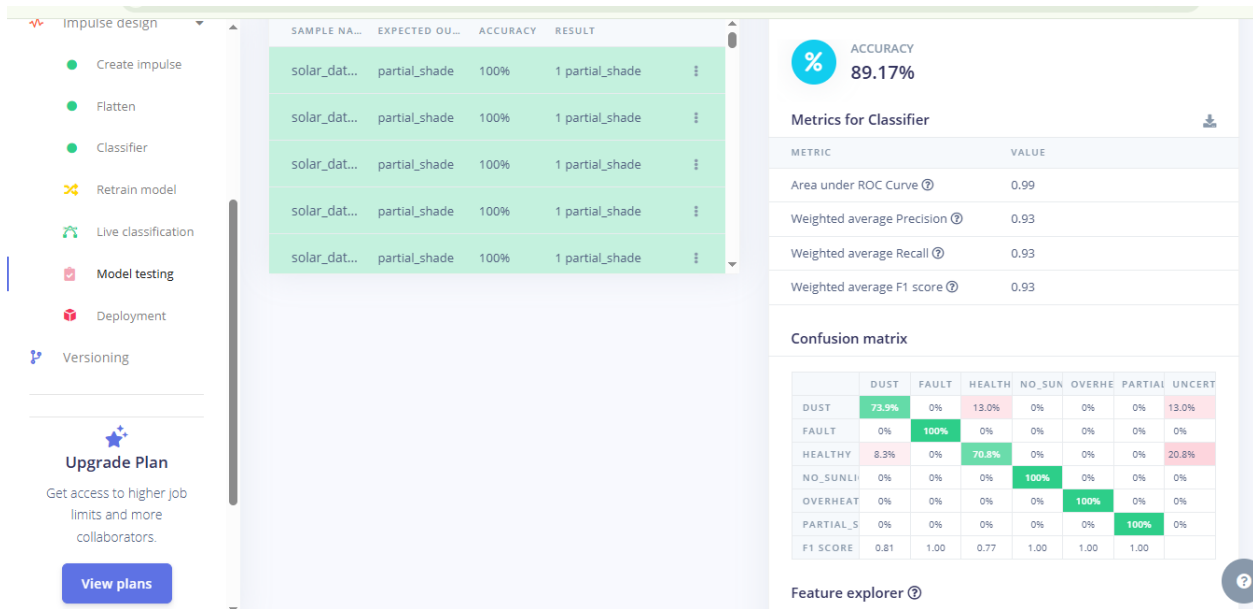


Figure 5.2: Held-out test-set performance — 89.17% accuracy, 0.99 AUC, with Fault, No Sunlight, Overheating, and Partial Shade all classified perfectly.

5.5 Deployment

The final model was exported from Edge Impulse as an Arduino library (Float32 / unoptimized, to preserve accuracy), with the EON Compiler enabled for reduced RAM/flash usage without affecting accuracy. On-device profiling reported an inference time of approximately 3 milliseconds, 1.4KB of peak RAM usage, and 13.9KB of flash usage — comfortably within the ESP32's available resources, leaving ample headroom for the WiFi/MQTT stack and other firmware logic.

6. Software Integration

6.1 On-Device Inference

The exported Edge Impulse library was integrated into the sensor-reading Arduino sketch. After each round of sensor reads, the four feature values (voltage, current, temperature, light) are passed into the model's `run_classifier()` function in the same order used during training, and the class with the highest confidence score is selected as the predicted condition.

6.2 MQTT Communication

The ESP32 publishes a JSON payload containing the raw sensor readings and the AI prediction to a HiveMQ Cloud MQTT broker over a TLS-secured connection (port 8883), authenticated using broker credentials. A cloud-hosted broker was chosen since the Node-RED dashboard runs on a separate laptop belonging to a project teammate; this removes the need for both devices to be on the same local network.

The published topic (solar/telemetry) and JSON field names (voltage, current, power, temp, light, status, confidence) are matched exactly to the structure expected by the Node-RED flow's processing function.

6.3 Node-RED Dashboard

The dashboard was built using Node-RED with the node-red-dashboard palette. An MQTT-in node subscribes to the telemetry topic, feeding a function node that parses the payload and derives dashboard-ready values (including a colour-coded status indicator and a derived Power Generation Active/Inactive flag based on measured voltage). A dropdown selector allows the user to switch between Live Parameters, AI Health Prediction, and System Status views, while a live line chart of all five sensor values is displayed continuously beneath the selected view, regardless of which option is chosen.

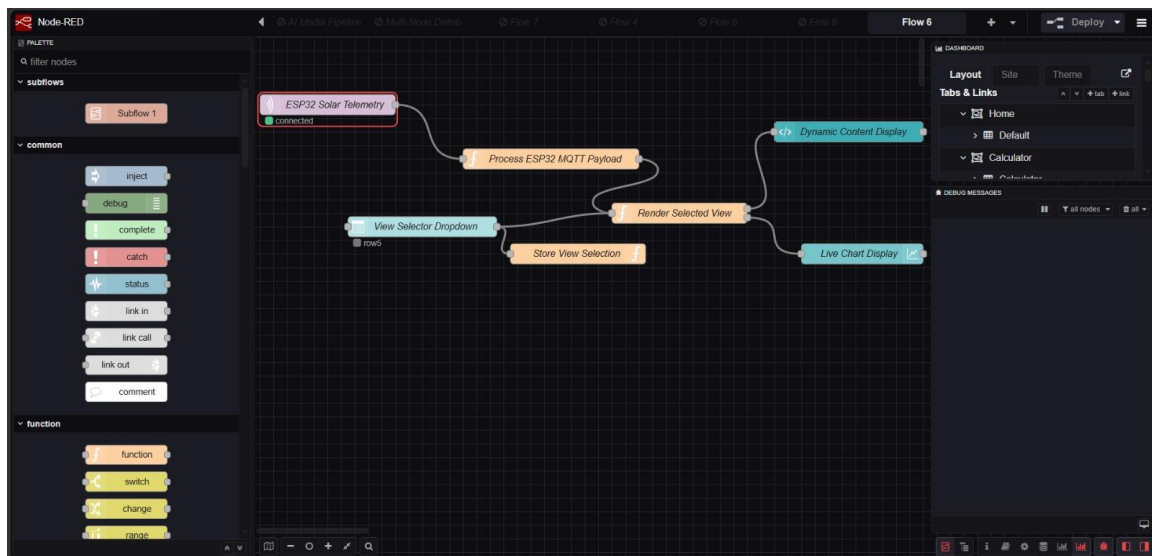


Figure 6.1: Node-RED flow structure — MQTT input, payload processing, dropdown-based view selection, and dashboard rendering nodes.

The core logic of the dashboard is implemented in two JavaScript function nodes within this flow: one that parses and restructures the incoming MQTT payload, and one that renders the currently selected dashboard view. The full source of both is included in Appendix B.

7. Results and Testing

7.1 Live System Demonstration

The complete system was tested live across two laptops: one running the Arduino IDE with the ESP32 connected via USB (uploading firmware and monitoring Serial output), and a second running the Node-RED dashboard in a browser, both communicating through the HiveMQ Cloud broker.

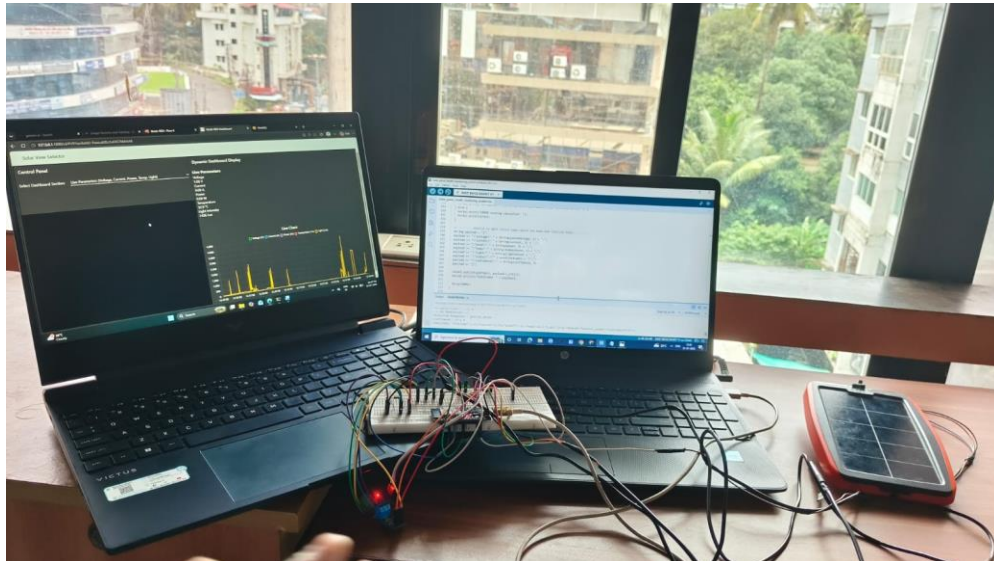


Figure 7.1: Live testing setup — ESP32 with breadboard-mounted sensors connected to the solar panel, Arduino IDE Serial Monitor (right) and the live Node-RED dashboard (left) updating in real time.

7.2 Serial Monitor Verification

The ESP32's Serial Monitor output was inspected directly to confirm that sensor readings, AI predictions, and the exact JSON payload being published over MQTT were all consistent with one another for each sample cycle.

```
Solar_panel_health_monitoring_system | Arduino IDE 2.3.2
File Edit Sketch Tools Help
DOIT ESP32 DEVKIT V1
Solar_panel_health_monitoring_system.ino
27 // ----- Pin Definitions -----
Output Serial Monitor x
Message (Enter to send message to 'DOIT ESP32 DEVKIT V1' on 'COM3') Both NL & CR 115200 baud

Voltage : 3.42 V
Estimated Current : 0.246 A
Estimated Power : 0.84 W
--- AI Prediction ---
Predicted Condition : overheating
Confidence : 78.2 %
Published: {"voltage":3.42,"current":0.246,"power":0.84,"temp":31.1,"light":28,"status":"overheating","confidence":78.2}
-----
Temperature : 31.12 C
Light : 36
Voltage : 3.51 V
Estimated Current : 0.253 A
Estimated Power : 0.89 W
--- AI Prediction ---
Predicted Condition : dust
Confidence : 52.7 %
Published: {"voltage":3.51,"current":0.253,"power":0.89,"temp":31.1,"light":36,"status":"dust","confidence":52.7}
-----
Temperature : 31.12 C
Light : 0
Voltage : 4.80 V
Estimated Current : 0.346 A
Estimated Power : 1.66 W
--- AI Prediction ---
Predicted Condition : healthy
Confidence : 84.0 %
Published: {"voltage":4.80,"current":0.346,"power":1.66,"temp":31.1,"light":0,"status":"healthy","confidence":84.0}
In 42, Col 40 DOIT ESP32 DEVKIT V1 on COM3
```

Figure 7.2: Serial Monitor output showing three consecutive sample cycles, each with raw sensor readings, the AI-predicted condition with confidence score, and the corresponding published MQTT JSON payload.

7.3 MQTT Message Verification

The HiveMQ Cloud web client was used to independently verify that the ESP32 was successfully publishing correctly formatted JSON messages to the expected topic.

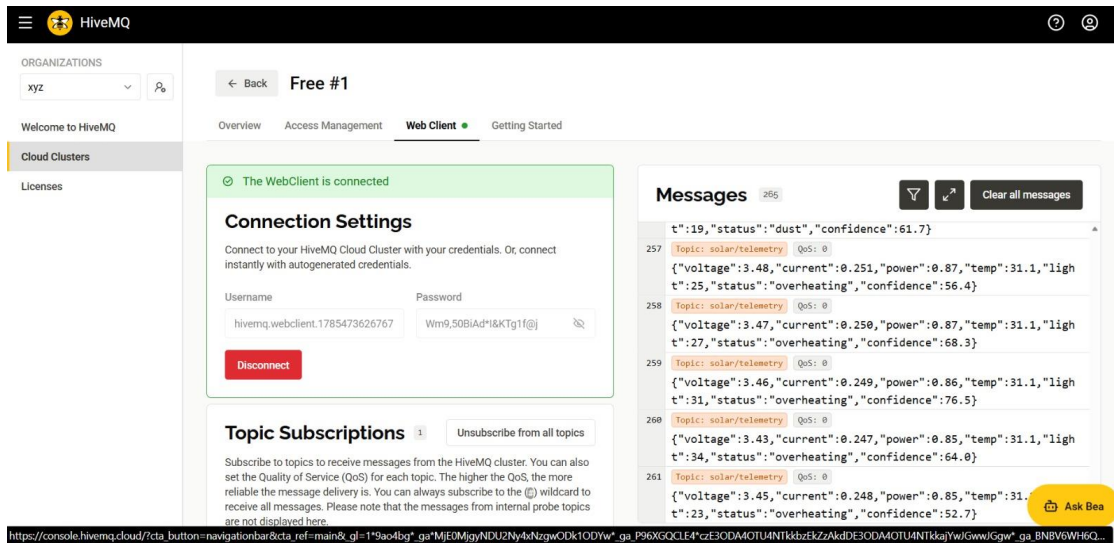


Figure 7.3: HiveMQ Cloud web client showing live incoming telemetry messages on the solar/telemetry topic.

7.4 Dashboard Output

The Node-RED dashboard successfully displayed live AI predictions with confidence scores and derived power-generation status, alongside overall system connectivity status. The AI Health Prediction view was tested against all six trained conditions during live operation, correctly reflecting the panel's simulated state in each case:

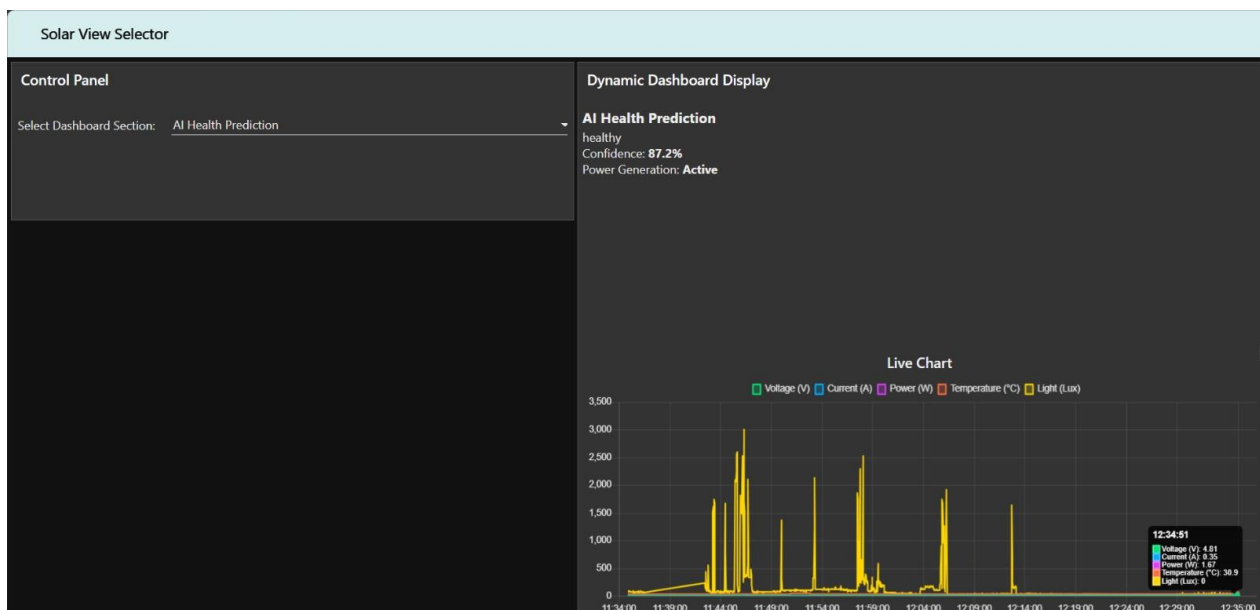


Figure 7.4(a): AI Health Prediction view showing a "Healthy" classification at 87.2% confidence.

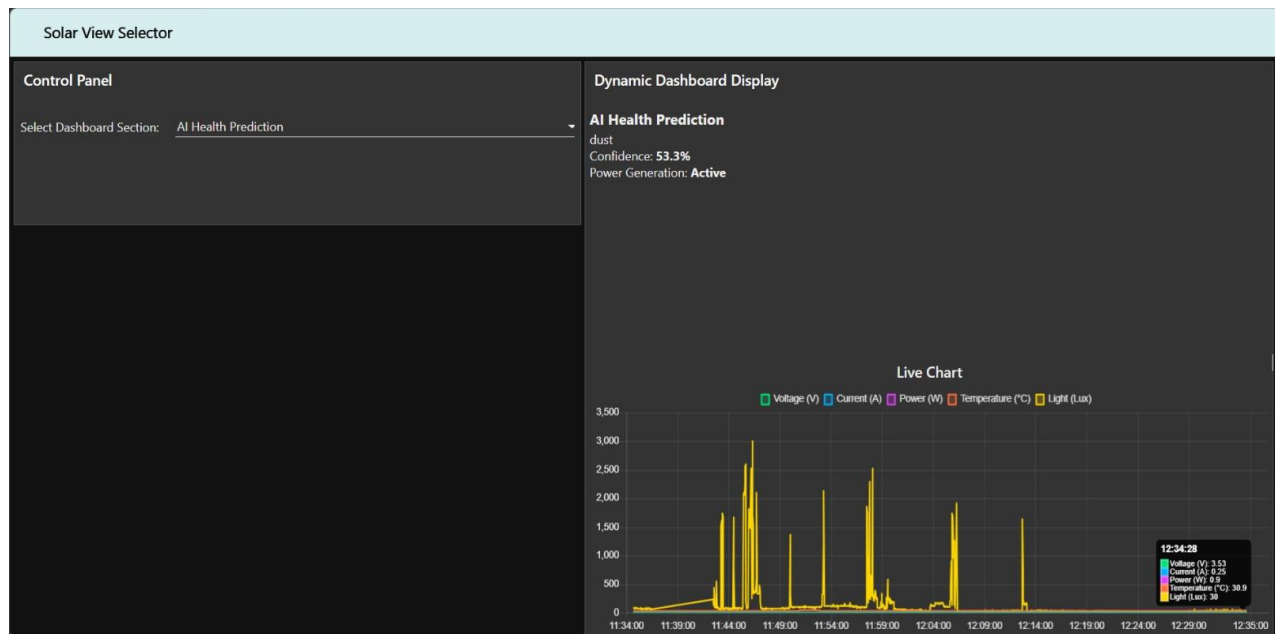


Figure 7.4(b): AI Health Prediction view showing a "Dust" classification at 53.3% confidence.

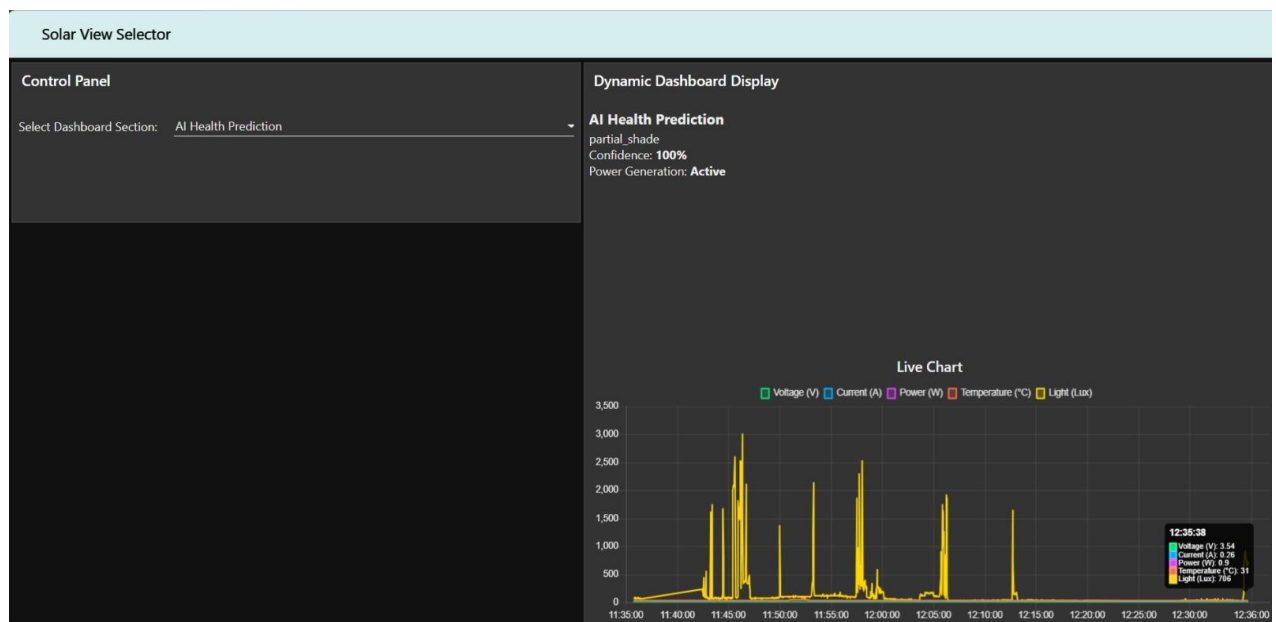


Figure 7.4(c): AI Health Prediction view showing a "Partial Shade" classification at 100% confidence.

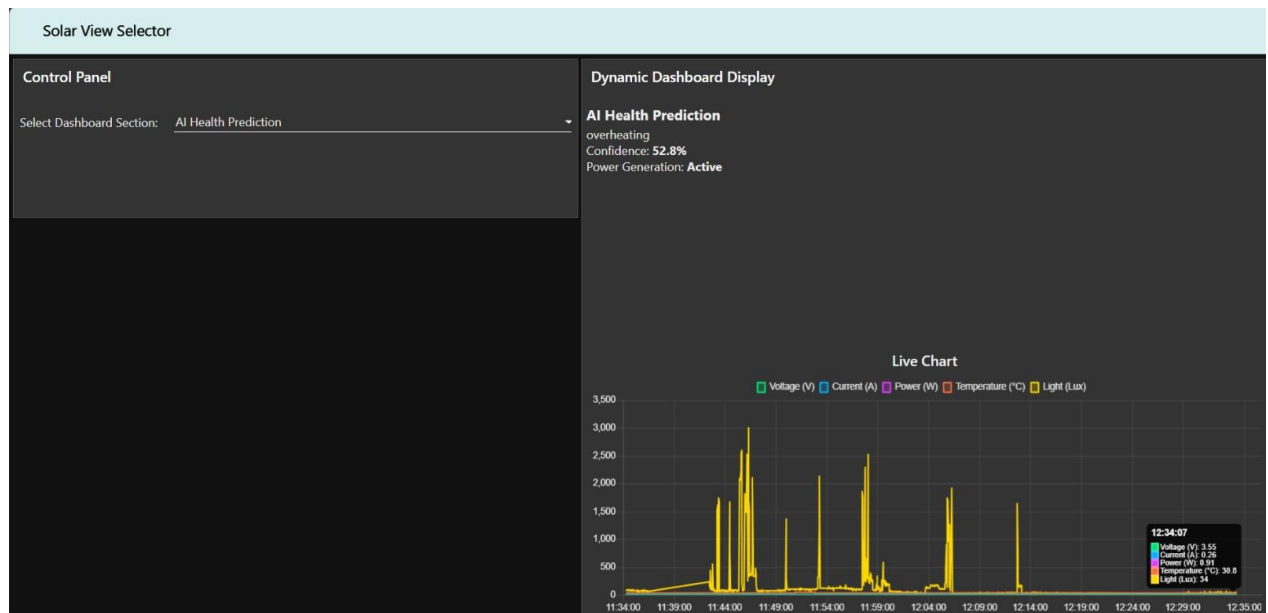


Figure 7.4(d): AI Health Prediction view showing an "Overheating" classification at 52.8% confidence.

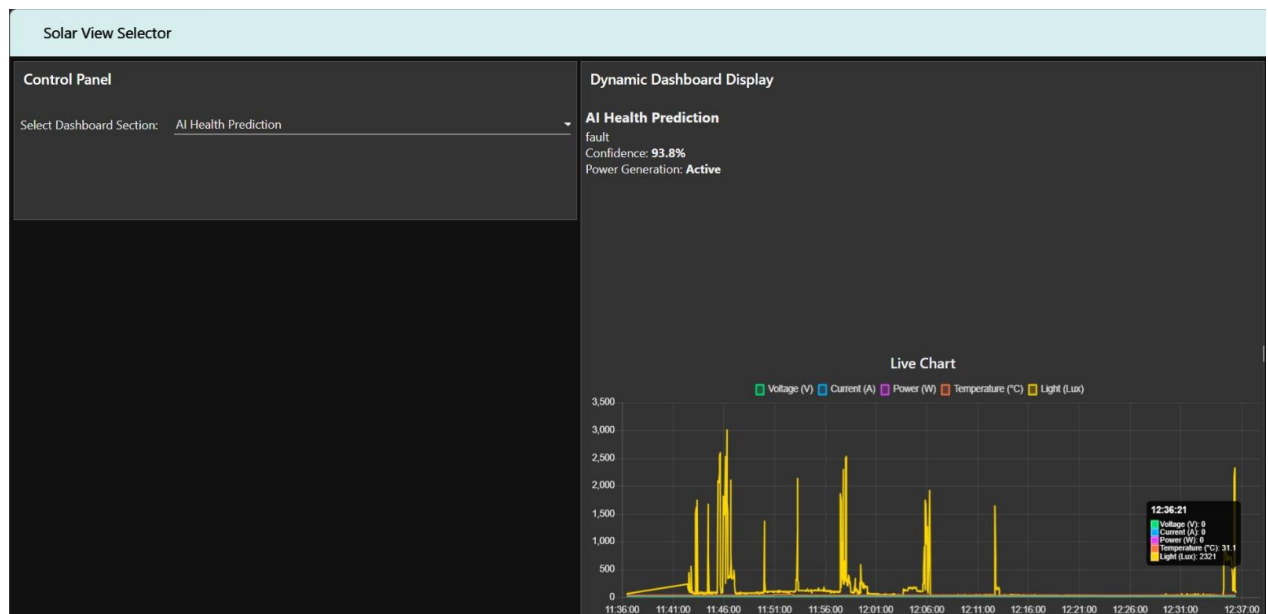


Figure 7.4(e): AI Health Prediction view showing a "Fault" classification at 93.8% confidence, with voltage and current both at zero.

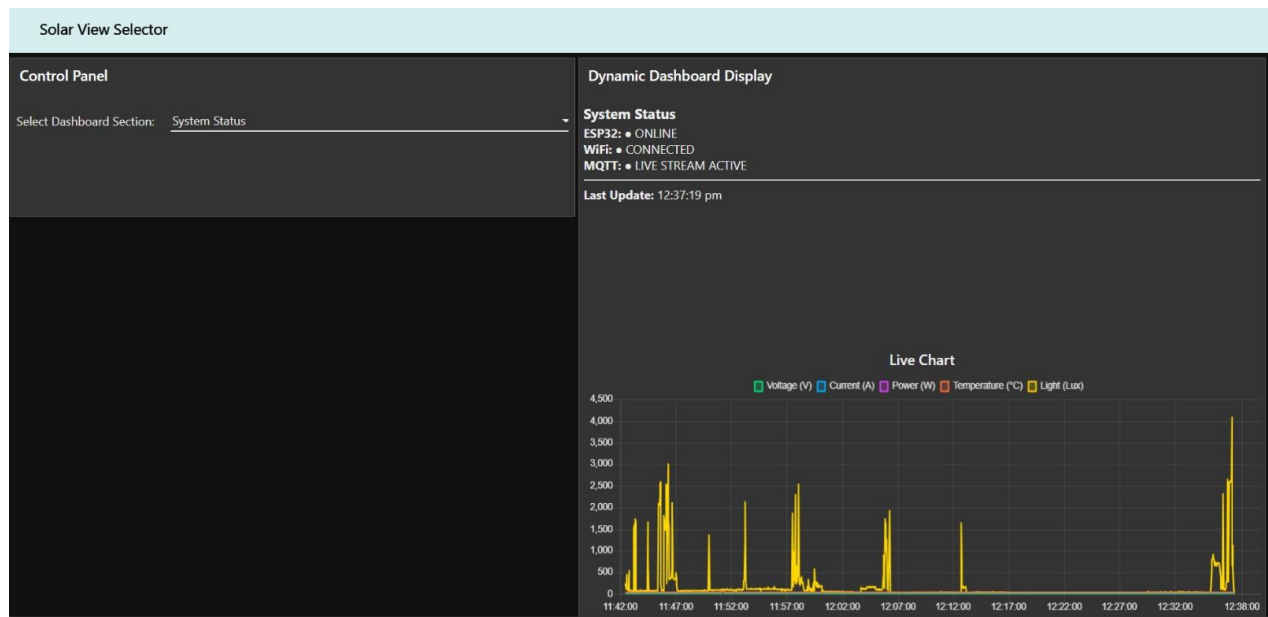


Figure 7.5: System Status view confirming ESP32, WiFi, and MQTT connectivity, alongside the continuously updating live chart.

7.5 Individual Parameter Trends

The Live Chart on the dashboard plots all five sensor values together by default. By toggling off the other series in the chart legend, each parameter's trend can be isolated for closer inspection.

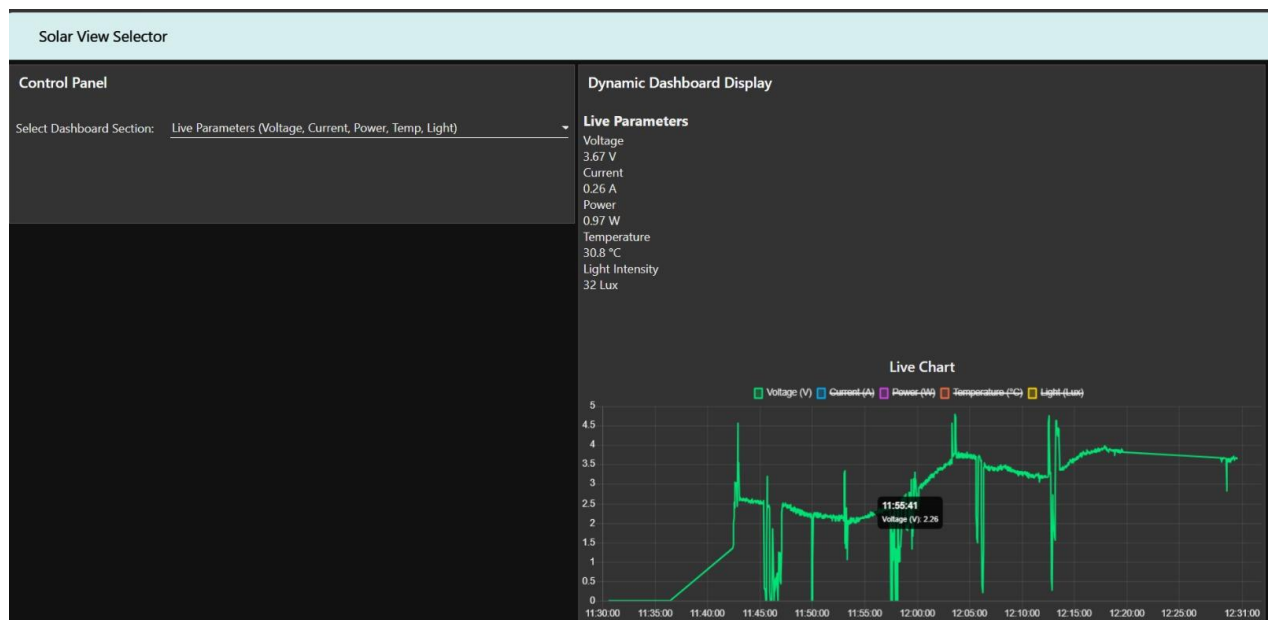


Figure 7.6(a): Voltage (V) trend isolated from the live chart, showing fluctuations as the panel was exposed to varying light conditions during testing.

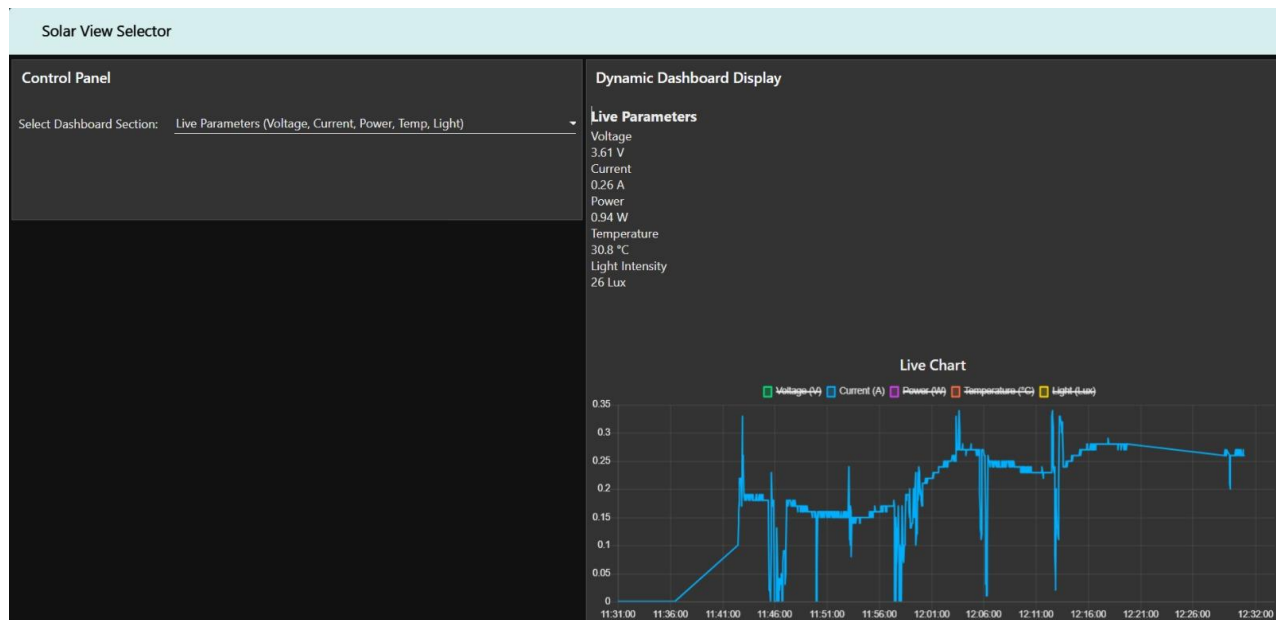


Figure 7.6(b): Current (A) trend isolated from the live chart, tracking proportionally with voltage as expected from the estimated-current calculation.

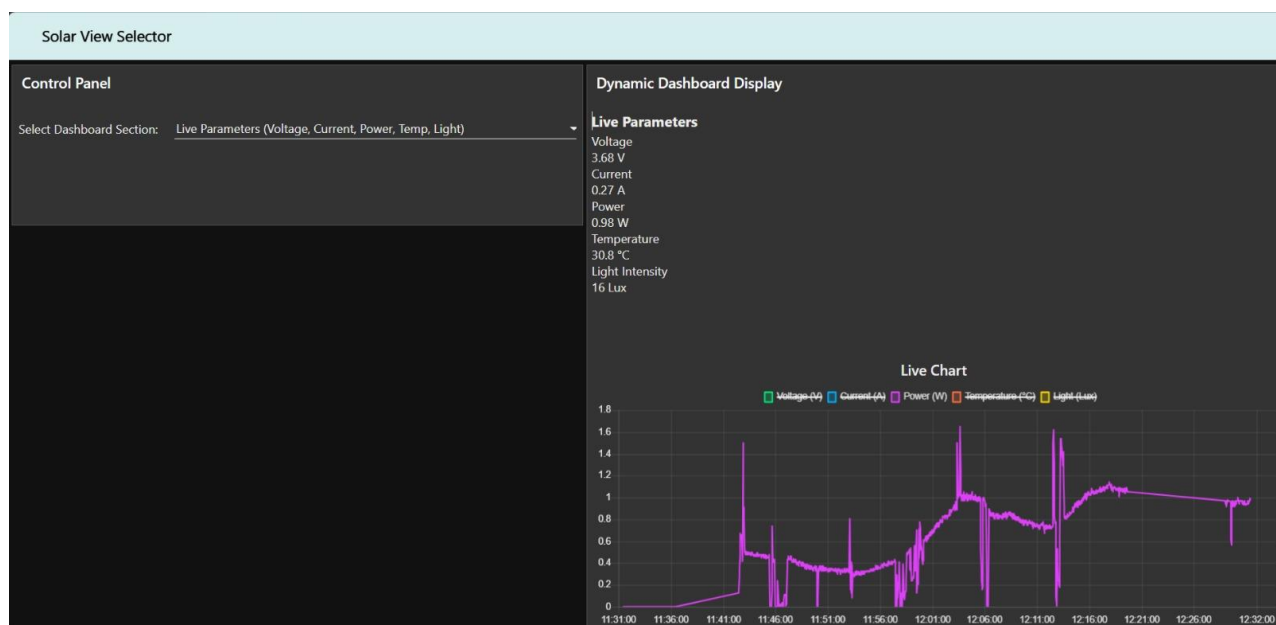


Figure 7.6(c): Power (W) trend isolated from the live chart, computed as the product of voltage and current.

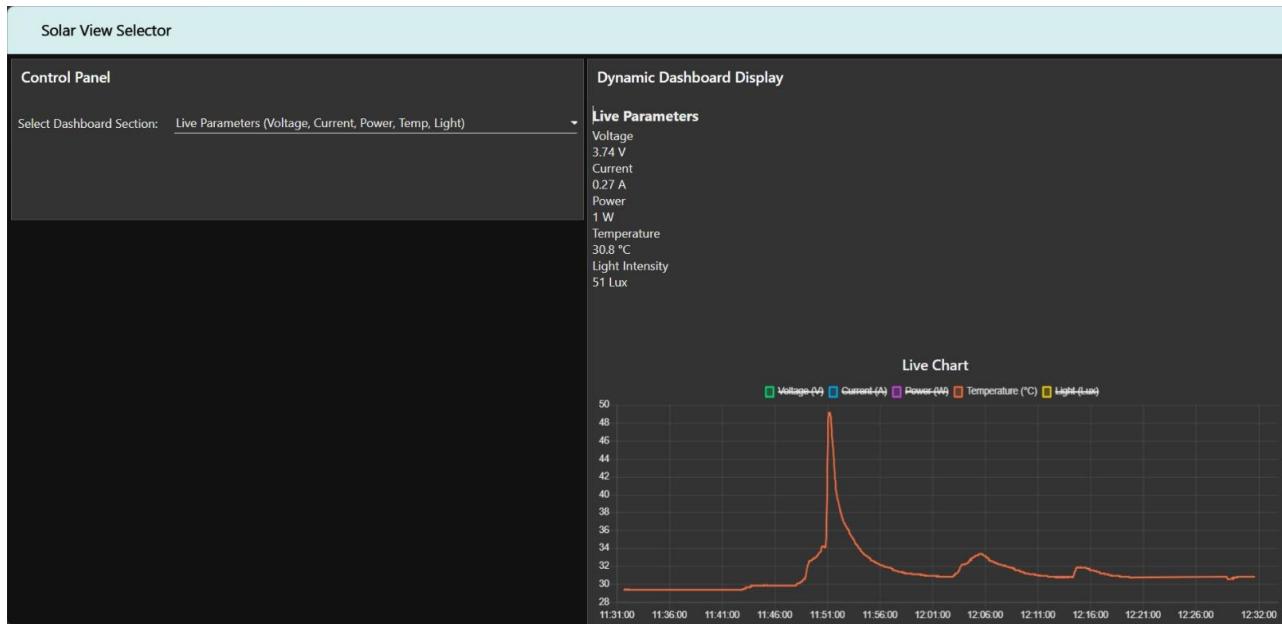


Figure 7.6(d): Temperature ($^{\circ}\text{C}$) trend isolated from the live chart, showing a sharp rise corresponding to the panel being manually warmed to simulate the Overheating condition.

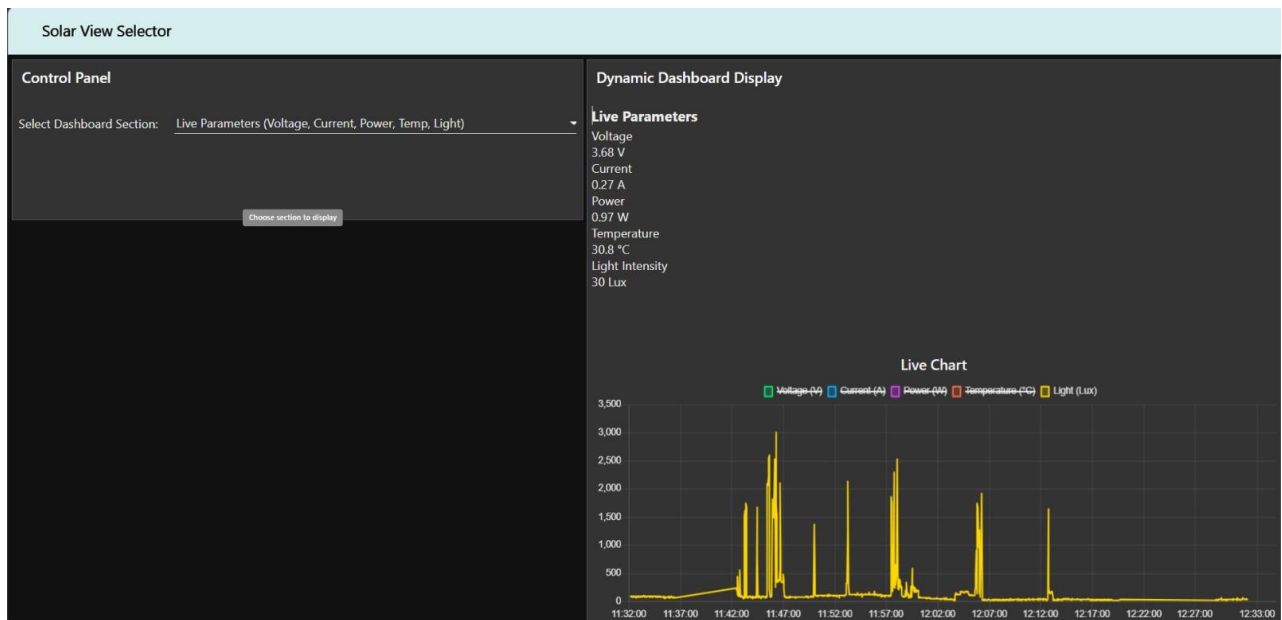


Figure 7.6(e): Light intensity (Lux) trend isolated from the live chart, showing sharp spikes corresponding to the torch/light source being moved closer to and away from the LDR during different test conditions.

7.6 Discussion

The final deployed model achieved strong, reliable performance on the safety-critical classes most relevant to fault detection — Fault, No Sunlight, Overheating, and Partial Shade all reached 100% test accuracy — while Dust and Healthy, which share more overlapping sensor signatures, achieved lower but still reasonable accuracy. Given that misclassifications between these two classes represent a relatively low-risk failure mode (a healthy panel being flagged as dusty, rather than a genuine fault going undetected), this performance profile represents a strong overall result for the project's scope.

8. Conclusion and Future Scope

This project successfully demonstrates an end-to-end Edge AI pipeline for solar panel health monitoring, from raw sensor acquisition through on-device machine learning inference to a live, cloud-connected dashboard. The final system reliably classifies six distinct panel operating conditions with 89.17% test accuracy, with the majority of conditions classified with perfect or near-perfect accuracy, and successfully streams live predictions to a remote dashboard over a cloud MQTT broker.

Future improvements could include replacing the estimated (voltage-derived) current value with an independently calibrated current sensor; collecting additional Healthy and Dust samples under more varied lighting conditions to further improve their classification accuracy; and extending the Node-RED dashboard with historical data logging and automated fault-alert notifications, building on the alert framework already outlined in the original system design.

9. References

1. Edge Impulse Documentation — <https://docs.edgeimpulse.com>
2. Espressif Systems, "ESP32 Technical Reference Manual"
3. Node-RED Documentation — <https://nodered.org/docs>
4. HiveMQ Cloud Documentation — <https://www.hivemq.com/docs>
5. Analog Devices, "ACS712 Current Sensor Datasheet"
6. Dallas Semiconductor, "DS18B20 Programmable Resolution Digital Thermometer Datasheet"

Appendix A: Key Source Code Excerpts

Selected excerpts from the final Arduino firmware are shown below, covering pin/WiFi configuration, MQTT setup, and the sensor-to-classifier-to-MQTT publish flow.

```
// ----- Pin Definitions -----
#define LDR_PIN      32
#define VOLTAGE_PIN  34
#define ONE_WIRE_BUS 4

// ----- DS18B20 -----
OneWire onewire(ONE_WIRE_BUS);
DallasTemperature sensors(&onewire);

// Voltage Sensor (0-25V Module)
const float VOLTAGE_DIVIDER_RATIO = 5.0;

// ----- WiFi Setup -----
void connectWiFi() {
  Serial.print("Connecting to WiFi");
  WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  Serial.println("\nWiFi connected. IP address: ");
  Serial.println(WiFi.localIP());
}
```

Pin definitions, DS18B20/voltage sensor setup, and WiFi connection routine.

```
// ----- MQTT Setup -----
void connectMQTT() {
  while (!client.connected()) {
    Serial.print("Connecting to HiveMQ Cloud...");
    if (client.connect(MQTT_CLIENT_ID, mqtt_user, mqtt_password)) {
      Serial.println("connected!");
    } else {
      Serial.print("failed, rc=");
      Serial.print(client.state());
      Serial.println(" retrying in 3 seconds...");
      delay(3000);
    }
  }
}

void setup()
{
  Serial.begin(115200);
  sensors.begin();

  connectWiFi();

  // NOTE: setInsecure() skips certificate validation - fine for a
  // student/demo project, but not recommended for production use.
  espClient.setInsecure();
}
```

MQTT connection handling and setup() routine.

```

// ----- Temperature -----
sensors.requestTemperatures();
float temperature = sensors.getTempCByIndex(0);

// ----- LDR -----
int lightValue = analogRead(LDR_PIN);

// ----- Voltage -----
int rawVoltage = analogRead(VOLTAGE_PIN);
float adcVoltage = rawVoltage * (3.3 / 4095.0);
float panelVoltage = adcVoltage * VOLTAGE_DIVIDER_RATIO;

if (panelVoltage < 0.20)
    panelVoltage = 0;

// ----- Estimated Current -----
float current = 0;
if (panelVoltage > 0) {
    current = (panelVoltage / 5.0) * 0.36;
    if (current > 0.36)
        current = 0.36;
}

// ----- Estimated Power -----
float power = panelVoltage * current;

```

Sensor reading, voltage scaling, and estimated current/power calculation.

```

// ----- Publish to MQTT (field names match the Node-RED function node) -----
String payload = "{";
payload += "\"voltage\":" + String(panelVoltage, 2) + ",";
payload += "\"current\":" + String(current, 3) + ",";
payload += "\"power\":" + String(power, 2) + ",";
payload += "\"temp\":" + String(temperature, 1) + ",";
payload += "\"light\":" + String(lightValue) + ",";
payload += "\"status\":" + predictedLabel + ",";
payload += "\"confidence\":" + String(confidence, 1);
payload += "}";

client.publish(pubTopic, payload.c_str());
Serial.println("Published: " + payload);

delay(2000);
}

```

Final MQTT JSON payload construction and publish call.

Appendix B: Node-RED Function Node Source Code

The Node-RED dashboard's core logic lives in two JavaScript function nodes within the flow. The first parses each incoming MQTT payload and restructures it into a state object used by all dashboard views; the second renders the HTML content for whichever view is currently selected and continuously forwards data to the live chart.

B.1 "Process ESP32 MQTT Payload" Function Node

```
const incoming = msg.payload;

// Ensure payload is a valid object
if (!incoming || typeof incoming !== 'object') {
  return null;
}

// Parse incoming metrics safely from ESP32
const voltage = +(incoming.voltage || 0).toFixed(2);
const current = +(incoming.current || 0).toFixed(2);
const power = +(incoming.power || (voltage * current)).toFixed(2);
const temp = +(incoming.temp || 0).toFixed(1);
const light = Math.floor(incoming.light || 0);

const status = incoming.status || "Healthy";
const alert = incoming.alert || "NO FAULT DETECTED";

let color = "#00E676";
if (status.includes("Dust")) color = "#FFB300";
else if (status.includes("Shade")) color = "#FFF176";
else if (status.includes("Temp") || status.includes("Overheating")) color = "#FF5252";
else if (status.includes("Fault")) color = "#FF1744";

const confidence = incoming.confidence || 99;
const timeStr = new Date().toLocaleTimeString();

// Power generation is derived from actual voltage unless explicitly provided
const genStatus = incoming.gen || (voltage > 0.2 ? "Active" : "Inactive");

const state = {
  row1: { voltage, current, power, temp, light },
  row2: { status, color, confidence, alert, gen: genStatus },
  row3: {
    panelHealth: status === "Healthy" ? "Normal" : status,
    battery: incoming.battery || "Connected",
    gen: genStatus
  },
  row4: { voltage, current, power, temp, light },
  row5: {
    esp: "ONLINE",
    wifi: "CONNECTED",
    mqtt: "LIVE STREAM ACTIVE",
    time: timeStr
  },
  row6: alert
};

flow.set('systemState', state);
return { payload: state };
```

B.2 "Render Selected View" Function Node

```
const selectedRow = flow.get('selectedRow') || "row2";
const state = flow.get('systemState');

if (!state) return null;

let content = "";

switch (selectedRow) {
  case "row1":
    content = `

### 


```